Fingerprint Sensor Testing Report

AS608 Fingerprint Sensor with ESP32 DevKit V1

Project:	IoT-Based Parliamentary Voting System
Test Module:	AS608 Fingerprint Sensor
Microcontroller:	ESP32 DevKit V1
Testing Platform:	Arduino IDE and Serial Monitor
Communication Protocol:	UART Serial Communication
Status:	Successfully Tested

1. Introduction

The purpose of this testing activity was to verify the functionality of the AS608 fingerprint sensor and establish reliable communication between the sensor and the ESP32 DevKit V1 microcontroller.

The AS608 fingerprint sensor was selected as a biometric identification component for the planned IoT-based parliamentary voting system. The system will use fingerprint verification to identify registered parliamentary members before allowing them to participate in voting sessions.

The initial testing focused on sensor connectivity, fingerprint detection, image processing, fingerprint enrollment, template storage, and verification of previously registered fingerprints. The successful completion of these tests provides a foundation for integrating biometric authentication into the larger IoT project.

2. Objectives

The primary objectives were:

1. Establish UART communication between the AS608 sensor and ESP32.
2. Verify that the sensor can detect a finger and capture a fingerprint image.
3. Process captured fingerprint images into fingerprint templates.
4. Enroll and save fingerprints using unique identification numbers.
5. Verify a fingerprint against previously stored templates.
6. Display meaningful diagnostic and verification messages through the Arduino Serial Monitor.
7. Identify and resolve problems encountered during initial testing.
8. Prepare a plan for integrating the sensor into the complete IoT voting system.

3. Hardware and Software Requirements

3.1 Hardware

Component	Purpose
ESP32 DevKit V1	Main microcontroller and future Wi-Fi connectivity
AS608 fingerprint sensor	Biometric fingerprint capture and matching
USB cable	Programming and serial communication
Connecting wires	Electrical connections between the ESP32 and sensor

Component	Purpose
Computer	Arduino IDE, programming, and testing

3.2 Software

- Arduino IDE
- Adafruit Fingerprint Sensor Library
- ESP32 board support package for Arduino IDE
- Arduino Serial Monitor

3.3 Communication Configuration

The ESP32 hardware UART interface was configured to communicate with the AS608 sensor at a baud rate of 57600. The Serial Monitor operated at 115200 baud to display diagnostic information and user commands.

4. Hardware Pin Configuration

The following connections were used during testing.

AS608 pin	ESP32 DevKit V1 pin	Function
GND	GND	Common ground
RX	GPIO 17 (TX2)	Receives serial data from ESP32
TX	GPIO 16 (RX2)	Transmits serial data to ESP32
3V3	Compatible power input	Sensor power, subject to module specifications
T-out	Not connected	Reserved until function is confirmed
T-3v	Not connected	Reserved until function is confirmed

The UART connections were crossed because the transmitting pin of one device must connect to the receiving pin of the other device.

Note: The exact supply voltage and the functions of the auxiliary pins must be verified against the documentation for the particular sensor module.

5. Testing Methodology

The testing was conducted incrementally to isolate potential hardware, communication, and software problems.

Test 1: Hardware Connection

The AS608 sensor was connected to the ESP32 using the specified UART pins and a common ground. The connections were checked before powering the hardware.

Purpose: Establish a suitable electrical connection between the microcontroller and fingerprint sensor.

Test 2: Sensor Communication

The Adafruit Fingerprint Sensor Library was used to initialize the UART connection and verify communication with the sensor. The program used the sensor verification function to determine whether the connected module responded successfully.

Expected result: A successful sensor connection message.

Test 3: Finger Detection

A diagnostic program repeatedly requested fingerprint images from the sensor. When a finger was placed on the scanning surface, the program displayed a detection message and attempted to capture an image.

Expected result: The Serial Monitor reports that a finger has been detected and an image captured.

Test 4: Fingerprint Image Processing

After capturing an image, the program converted the image into a fingerprint template using the sensor's image-processing functionality. The program reported whether image processing succeeded or returned an error.

Expected result: A successful fingerprint-processing message.

Test 5: Fingerprint Enrollment

A new fingerprint was enrolled by scanning the same finger twice. The sensor compared both scans and generated a fingerprint model. The model was then stored in the sensor's internal memory under a selected ID.

Expected result: A successful enrollment message and confirmation of the stored fingerprint ID.

Test 6: Fingerprint Verification

The enrolled finger was scanned again. The sensor searched its stored templates and returned a matching fingerprint ID when a match was found. An unregistered finger could also be tested to check the no-match response.

Expected result: Correct identification of an enrolled fingerprint or a no-match result for an unregistered finger.

Test 7: Memory Verification

The program queried the sensor to retrieve the number of stored fingerprint templates. This test checked whether enrolled templates remained available after the enrollment operation.

Expected result: The Serial Monitor displays the number of stored fingerprints.

5.1 Complete Source Code Used for Successful Testing

The following Arduino program was used to test the AS608 fingerprint sensor with the ESP32 DevKit V1. It supports fingerprint enrollment, verification, stored-template counting, and deletion.

Required library: Adafruit Fingerprint Sensor Library

UART configuration:

- ESP32 GPIO 16 (RX2) connected to AS608 TX.
- ESP32 GPIO 17 (TX2) connected to AS608 RX.
- UART baud rate: 57600.
- Serial Monitor baud rate: 115200.

```
#include <Adafruit_Fingerprint.h>

// ESP32 DevKit V1 + AS608 pin configuration
#define FP_RX 16  // ESP32 RX2 <- AS608 TX
#define FP_TX 17  // ESP32 TX2 -> AS608 RX

HardwareSerial FingerSerial(2);
Adafruit_Fingerprint finger(&FingerSerial);

// Wait until the finger is removed
void waitForFingerRemoval() {
  Serial.println("Please remove your finger.");

  while (finger.getImage() != FINGERPRINT_NOFINGER) {
    delay(100);
  }

  Serial.println("Finger removed successfully.");
}

// Capture and process a fingerprint
bool scanFingerprint(uint8_t slot) {
  Serial.println("Place your finger on the sensor...");

  while (true) {
    uint8_t result = finger.getImage();

    if (result == FINGERPRINT_OK) {
      Serial.println("FINGER DETECTED!");
      Serial.println("Fingerprint image captured.");
      break;
    }

    if (result == FINGERPRINT_NOFINGER) {
      delay(100);
      continue;
    }

    Serial.print("Image capture error: ");
    Serial.println(result);
    return false;
  }

  Serial.println("Processing fingerprint...");

  uint8_t result = finger.image2Tz(slot);

  if (result == FINGERPRINT_OK) {
    Serial.println("Fingerprint processed successfully!");
    return true;
  }

  if (result == FINGERPRINT_IMAGEMESS) {
    Serial.println("Image too messy. Try again.");
  } else if (result == FINGERPRINT_FEATUREFAIL ||
             result == FINGERPRINT_INVALIDIMAGE) {
    Serial.println("Fingerprint features not detected.");
  } else {
    Serial.print("Processing error: ");
    Serial.println(result);
  }
```

```
  return false;
}

// Enroll a new fingerprint
void enrollFingerprint() {
  Serial.println("\n========== ENROLL FINGERPRINT ==========");
  Serial.println("Enter a fingerprint ID from 1 to 127:");

  while (Serial.available()) Serial.read();

  while (!Serial.available()) {
    delay(10);
  }

  int id = Serial.parseInt();

  while (Serial.available()) Serial.read();

  if (id < 1 || id > 127) {
    Serial.println("Invalid ID. Enter a number from 1 to 127.");
    return;
  }

  // Check whether the selected ID is already occupied
  if (finger.loadModel(id) == FINGERPRINT_OK) {
    Serial.println("This ID already contains a fingerprint.");
    Serial.println("Choose another ID or delete this one first.");
    return;
  }

  Serial.print("Starting enrollment for ID: ");
  Serial.println(id);

  // First scan
  Serial.println("\n--- FIRST SCAN ---");

  if (!scanFingerprint(1)) {
    waitForFingerRemoval();
    return;
  }

  waitForFingerRemoval();

  // Second scan of the same finger
  Serial.println("\n--- SECOND SCAN ---");
  Serial.println("Scan the SAME finger again.");

  if (!scanFingerprint(2)) {
    waitForFingerRemoval();
    return;
  }

  // Generate a fingerprint model from both scans
  Serial.println("Comparing fingerprint scans...");

  uint8_t result = finger.createModel();

  if (result == FINGERPRINT_OK) {
    Serial.println("Both scans match!");
  } else if (result == FINGERPRINT_ENROLLMISMATCH) {
    Serial.println("ERROR: Scans do not match.");
    Serial.println("Please enroll again using the same finger.");
```

```
    waitForFingerRemoval();
    return;
  } else {
    Serial.print("Fingerprint model error: ");
    Serial.println(result);
    waitForFingerRemoval();
    return;
  }

  // Save the model in the sensor's internal memory
  Serial.println("Saving fingerprint to sensor memory...");

  result = finger.storeModel(id);

  if (result == FINGERPRINT_OK) {
    Serial.println("\n*** FINGERPRINT SAVED SUCCESSFULLY! ***");
    Serial.print("Saved fingerprint ID: ");
    Serial.println(id);
  } else {
    Serial.print("Failed to save fingerprint. Error: ");
    Serial.println(result);
  }

  waitForFingerRemoval();
}

// Verify a fingerprint against saved templates
void verifyFingerprint() {
  Serial.println("\n========== FINGERPRINT VERIFICATION ==========");

  if (!scanFingerprint(1)) {
    waitForFingerRemoval();
    return;
  }

  Serial.println("Searching saved fingerprints...");

  uint8_t result = finger.fingerSearch();

  if (result == FINGERPRINT_OK) {
    Serial.println("\n*** MATCH FOUND! ***");
    Serial.print("Fingerprint ID: ");
    Serial.println(finger.fingerID);

    Serial.print("Confidence score: ");
    Serial.println(finger.confidence);

    Serial.println("Verification successful!");
  } else if (result == FINGERPRINT_NOTFOUND) {
    Serial.println("\nNO MATCH FOUND.");
    Serial.println("This finger is not enrolled.");
  } else {
    Serial.print("Fingerprint search error: ");
    Serial.println(result);
  }

  waitForFingerRemoval();
}

// Count stored fingerprints
void countFingerprints() {
  Serial.println("\nChecking sensor memory...");
```

```
  if (finger.getTemplateCount() == FINGERPRINT_OK) {
    Serial.print("Total saved fingerprints: ");
    Serial.println(finger.templateCount);
  } else {
    Serial.println("Could not read fingerprint count.");
  }
}

// Delete a stored fingerprint by ID
void deleteFingerprint() {
  Serial.println("\nEnter the fingerprint ID to delete:");

  while (Serial.available()) Serial.read();

  while (!Serial.available()) {
    delay(10);
  }

  int id = Serial.parseInt();

  while (Serial.available()) Serial.read();

  if (id < 1 || id > 127) {
    Serial.println("Invalid ID.");
    return;
  }

  uint8_t result = finger.deleteModel(id);

  if (result == FINGERPRINT_OK) {
    Serial.println("Fingerprint deleted successfully.");
  } else {
    Serial.print("Could not delete fingerprint. Error: ");
    Serial.println(result);
  }
}

// Display available commands
void showMenu() {
  Serial.println("\n==================================");
  Serial.println("       AS608 FINGERPRINT MENU");
  Serial.println("==================================");
  Serial.println("E = Enroll new fingerprint");
  Serial.println("V = Verify fingerprint");
  Serial.println("C = Count saved fingerprints");
  Serial.println("D = Delete fingerprint by ID");
  Serial.println("M = Show menu");
  Serial.println("==================================");
}

// Initialize the sensor
void setup() {
  Serial.begin(115200);
  delay(1000);

  FingerSerial.begin(57600, SERIAL_8N1, FP_RX, FP_TX);
  finger.begin(57600);

  Serial.println("\nESP32 + AS608 Fingerprint System");
  Serial.println("Connecting to sensor...");
```

```
  if (!finger.verifyPassword()) {
    Serial.println("ERROR: Sensor communication failed!");
    Serial.println("Check wiring, power and baud rate.");

    while (true) {
      delay(1000);
    }
  }

  Serial.println("Sensor connected successfully!");

  countFingerprints();
  showMenu();
}

// Main program loop
void loop() {
  if (!Serial.available()) {
    return;
  }

  char command = Serial.read();

  while (Serial.available()) Serial.read();

  switch (command) {
    case 'E':
    case 'e':
      enrollFingerprint();
      showMenu();
      break;

    case 'V':
    case 'v':
    case 'T':
    case 't':
      verifyFingerprint();
      showMenu();
      break;

    case 'C':
    case 'c':
      countFingerprints();
      showMenu();
      break;

    case 'D':
    case 'd':
      deleteFingerprint();
      showMenu();
      break;

    case 'M':
    case 'm':
      showMenu();
      break;

    default:
      Serial.println("Unknown command. Type M to show menu.");
      break;
  }
}
```

Code execution procedure

1. Install the Adafruit Fingerprint Sensor Library in Arduino IDE.
2. Connect the AS608 to the ESP32 according to the pin configuration.
3. Select the ESP32 Dev Module board and the correct serial port.
4. Upload the program.
5. Open Serial Monitor at 115200 baud and select Newline as the line ending.
6. Enter E to enroll a fingerprint, then provide an ID from 1 to 127.
7. Scan the same finger twice as prompted.

8 Enter `V` to verify the enrolled fingerprint, `C` to count stored templates, or `D` to delete a template by ID.

Expected result

The program should report successful sensor communication, fingerprint detection, image processing, enrollment, and verification. For a successful match, it displays the stored fingerprint ID and the confidence score returned by the sensor.

Testing conclusion: This program served as the functional test implementation for the AS608 fingerprint sensor. The reported successful operation establishes that the sensor can perform basic biometric enrollment and matching with the ESP32 DevKit V1. Further testing is required to establish long-term reliability and security for the complete IoT parliamentary voting system.

6. Problems Encountered and Solutions

Problem 1: Finger Detection Was Not Initially Visible

Observed problem: During the initial enrollment attempt, the expected indication of finger detection was not sufficiently clear. It was difficult to determine whether the sensor was waiting for a finger, capturing an image, or failing to process the image.

Cause: The initial program provided limited diagnostic feedback and waited for certain sensor responses without clearly reporting each stage. The absence of a visible message did not necessarily mean that the sensor was defective.

Solution implemented:

- Developed a diagnostic sketch to check sensor communication.
- Added explicit messages for finger detection.
- Added confirmation messages for successful image capture.
- Added messages for image-processing errors.
- Added a prompt when the finger needed to be removed.

Result: The diagnostic program successfully detected fingerprints and reported the processing status.

Problem 2: Fingerprint Enrollment Required Two Scans

Observed problem: Fingerprint enrollment could not be completed reliably without capturing the same finger twice and comparing the two samples.

Cause: The enrollment procedure requires two fingerprint samples to generate a matching model. The first and second scans must be processed into separate template buffers.

Solution implemented:

- Used template buffer 1 for the first scan.
- Used template buffer 2 for the second scan.
- Compared the scans using the sensor's model-generation function.
- Stored the generated model only after the comparison succeeded.

Result: The enrollment process worked successfully when the same finger was scanned twice.

Problem 3: Fingerprint Identification Needed Explicit Feedback

Observed problem: A captured fingerprint needed to be distinguished from a successfully verified fingerprint.

Cause: Image capture, template processing, enrollment, and identification are separate operations. Capturing an image alone does not prove that the finger matches an enrolled user.

Solution implemented:

- Added a search operation against the sensor's stored templates.
- Displayed the matched fingerprint ID.
- Displayed the confidence score returned by the sensor.
- Added a separate message for fingerprints that were not found.

Result: The program could report successful matches and no-match outcomes separately.

Problem 4: Sensor Memory Needed Verification

Observed problem: It was necessary to confirm that fingerprint templates were saved inside the sensor rather than existing only temporarily in program memory.

Solution implemented:

- Used the sensor's template-storage function to save the fingerprint under an ID.
- Queried the sensor's stored-template count.
- Used the verification function to search previously enrolled templates.

Result: The completed test program successfully supported fingerprint storage, counting, and verification.

Important distinction: These were the problems encountered during the development and troubleshooting process. The exact underlying cause of each initial issue was not established independently unless confirmed by the diagnostic output.

7. Final Testing Results

Test case	Expected outcome	Final status
ESP32-to-sensor communication	Sensor responds correctly	Passed
Finger detection	Finger is detected	Passed
Image capture	Fingerprint image is captured	Passed
Image processing	Template processing succeeds	Passed
Fingerprint enrollment	New template is saved	Passed
Fingerprint verification	Enrolled finger is matched	Passed
No-match handling	Unregistered finger is rejected	Supported by code
Stored-template count	Number of templates is displayed	Supported by code
Fingerprint deletion	Selected template can be deleted	Supported by code

The user confirmed that the fingerprint system was working completely successfully after troubleshooting. The results above reflect the reported working status of the system. Individual test cases should be recorded as formally passed only after their outcomes have been observed and documented.

8. Final Outcome

The AS608 fingerprint sensor was successfully integrated with the ESP32 DevKit V1 for basic biometric enrollment and identification.

The completed test program provides:

- Fingerprint detection and image capture.
- Fingerprint template processing.
- Enrollment using a user-selected ID.
- Storage of fingerprint templates in sensor memory.
- Matching against previously enrolled fingerprints.
- Reporting of the matched ID and confidence score.
- Counting and deletion of stored templates.
- Diagnostic messages to support troubleshooting.

The sensor performs local fingerprint-template storage and matching, reducing the need to transmit raw fingerprint images to an external server for routine verification.

However, the tests do not yet establish biometric accuracy, false-acceptance rate, false-rejection rate, resistance to spoofing, or suitability for a production security system.

9. Next Plan: Integration into the Full IoT Parliamentary Voting System

The next development phase is to integrate the tested fingerprint sensor into the planned IoT-based parliamentary voting system. The objective is to use the fingerprint ID as a link to a registered member record and enforce voting eligibility through a centralized voting application.

Phase 1: Hardware Integration

Integrate the AS608 sensor with the ESP32 DevKit V1 alongside the planned display or status indicators, buttons, and buzzer.

1. Keep the verified UART pin configuration.
2. Add status LEDs or a buzzer to indicate successful and failed authentication.
3. Display prompts for scanning and verification.
4. Test continuous operation and repeated fingerprint scans.
5. Check power stability when all peripherals are connected.

Phase 2: Member Registration

Develop a web-based administration interface to register parliamentary members. Each member record should contain a unique member ID, name, registration status, and associated fingerprint template ID.

Fingerprint enrollment should be performed by authorized personnel. The application should prevent accidental reuse of fingerprint IDs and provide a controlled process for replacing or deactivating templates.

Phase 3: ESP32 and Server Communication

Connect the ESP32 to the local Wi-Fi network and develop a secure communication interface with the voting server. The server should maintain member records, bills, voting sessions, vote records, and audit logs.

The ESP32 should send the matched fingerprint ID and device information to the server for eligibility checks. It should not send raw fingerprint images during normal verification. Communication should use authenticated requests, encrypted transport such as HTTPS, and appropriate protection against replayed requests.

Phase 4: Voting Session Management

Implement the workflow for creating and approving a voting session. The authorized host should create a bill or proposal, set the voting question and session schedule, and submit the session for approval. Once

approved, the session should become available to eligible members.

The ESP32 should accept voting actions only when the server confirms that the session is active.

Phase 5: Member Authentication and Voting

The proposed voting process is:

1. A member places a finger on the AS608 sensor.
2. The sensor searches its enrolled fingerprint templates.
3. The ESP32 receives the matched fingerprint ID.
4. The server verifies the member's registration and eligibility.
5. The system checks whether the member has already voted in that session.
6. The member selects YES, NO, or another option permitted by the bill.
7. The server validates and records the vote.
8. The device displays a confirmation message.

The server must prevent duplicate votes and unauthorized voting. Fingerprint authentication establishes identity; it must not be treated as proof that the person is eligible to vote in every session.

Phase 6: Vote Integrity and Audit Logging

Develop safeguards for the voting process:

- Unique vote records and server-side duplicate prevention.
- Session locking and controlled session closure.
- Reliable handling of network failures.
- Authenticated host and administrator actions.
- Audit logs for session creation, approval, and closure.
- Secure backups and restricted access to member records.

If the goal is secret-ballot voting, identity verification must be separated from the stored vote choice. The system should not expose a direct member-to-vote-choice association to ordinary administrators.

Phase 7: Full-System Testing

Test the integrated system under realistic conditions. Planned scenarios include:

- Correct fingerprint and eligible member.
- Correct fingerprint but inactive member.
- Fingerprint not enrolled.
- Inactive or closed voting session.
- Member attempting to vote twice.
- Wi-Fi disconnection during a voting operation.
- Server unavailability.
- Device restart during a session.
- Unauthorized host or administrator access.
- Concurrent requests from multiple devices.

Testing should verify that failed or incomplete requests do not create duplicate or invalid vote records.

Phase 8: Performance and Security Evaluation

Measure the system's fingerprint-verification time, server-response time, end-to-end voting latency, and behavior under network interruption.

Evaluate biometric performance separately using an appropriately designed test protocol. A limited functional test is insufficient to establish production-grade biometric reliability.

Before deployment, assess sensor spoofing risks, device tampering, secure firmware updates, network security, privacy, and independent auditability.

10. Proposed Future Architecture

The intended architecture is:

AS608 Fingerprint Sensor -> ESP32 DevKit V1 -> Wi-Fi Network -> Secure Voting API -> Central Database -> Host Dashboard and Public Results Interface

The fingerprint sensor handles local template matching. The ESP32 manages the device interface and communicates with the server. The server controls registration, session eligibility, vote validation, and result calculation.

The web dashboard provides administrative functions, while the public interface displays approved voting information and results according to the project's transparency and ballot-secrecy requirements.

11. Conclusion

The initial fingerprint sensor testing was completed successfully. The AS608 sensor now communicates with the ESP32 DevKit V1, detects fingerprints, processes captured images, enrolls templates, and verifies previously registered fingerprints.

The troubleshooting process highlighted the importance of clear serial diagnostics, correct UART configuration, separate template buffers during enrollment, and explicit feedback during fingerprint matching.

The next stage is to develop a complete IoT parliamentary voting system in which fingerprint authentication is integrated with member registration, voting-session management, server-side vote validation, and reliable result reporting.

This test represents a successful proof of concept for the biometric subsystem. Further integration, performance evaluation, and security testing will be necessary before the complete system can be considered suitable for operational parliamentary voting.